14

INTRODUCTION TO ROOTKITS

Rootkits are malware variants that specialize in hiding their presence on a host by first obtaining low-level access to the victim system. The name *rootkit* originates from Unix, where the *root* user has the highest level of privileges the system allows. Rootkits use several evasion methods, such as intercepting and modifying communication between kernel and user space and directly tampering with data structures in kernel memory, to hide from endpoint defenses and investigation tools.

This chapter provides an introductory overview of kernel-based rootkits and some of the techniques they use to evade defenses and manipulate a system. While not an exhaustive resource on rootkits or rootkit analysis, this chapter covers some of the tactics to be on the lookout for when you're investigating low-level malware.

Rootkit Fundamentals

There are many reasons why a malware author might use rootkit components:

Persistence and survivability

Since rootkits exist in kernel space and have low-level system access, they can persist after reboots and in strong, well-defended environments. Bootkits, an advanced form of rootkit we'll discuss later in the chapter, reside at the firmware layer and therefore have even greater persistence.

Defense circumvention

Some rootkits actively tamper with and blind endpoint defenses such as EDR and anti-malware. Such rootkits can also hide and protect their files and processes from investigators by redirecting function calls, for example.

Low-level access to devices and drivers

Some rootkits intercept requests and commands to and from kernel drivers and hardware. One example is Moriya (see the May 2021 article "Operation TunnelSnake" at ***https://securelist.com/operation-tunnelsnake-and-moriya-rootkit/101831/***), which intercepts, manipulates, and hides network traffic to and from the infected host.

Because rootkits reside in kernel space and work by manipulating kernel elements, let's take a closer look at what these components are before discussing how rootkits take advantage of them.

Kernel Modules and Drivers

Kernel modules are binary files containing code and data that extend the kernel's functionalities. They can be loaded at system boot-up or on demand. *Kernel drivers* are a specific type of kernel module that interact with system hardware. There are different types of kernel drivers:

Device drivers

Perhaps the most common type of kernel driver, device drivers provide an interface between Windows and the underlying hardware devices of the system, such as keyboards, mice, and printers. They interact with system hardware either directly or indirectly.

Filter drivers

As their name suggests, filter drivers "filter" IO communication destined for other drivers, intercepting and potentially modifying it. These drivers add functionality to other drivers or to the system at large, and they also enable capabilities such as logging and monitoring. Some malicious actors load filter drivers in the kernel to take advantage of these benefits, as you'll see later.

Minifilter drivers

Similar to filter drivers, minifilter drivers filter IO operations and were introduced in more modern versions of Windows to improve performance and simplify development and compatibility. These drivers can also be abused by malicious actors.

NOTE

Even though all kernel drivers are modules, not all kernel modules are drivers. However, for simplicity's sake, I'll use the terms module *and* driver *interchangeably in this chapter.*

You can view loaded kernel modules in Windows using a Process Manager–like tool such as Process Hacker, as shown in **Figure 14-1**.

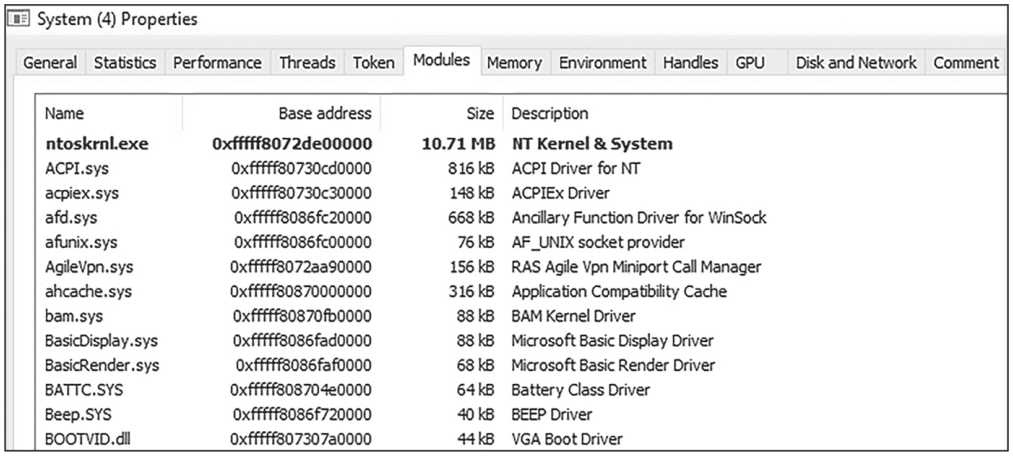

Figure 14-1: Viewing loaded kernel modules in Process Hacker

To view loaded kernel modules in Process Hacker, right-click the system process, select **Properties**, and then select the **Modules** tab. In **Figure 14-1**, you can see some of the kernel drivers installed on my system, such as Advanced Configuration and Power Interface (ACPI) drivers and display drivers such as the VGA boot driver.

Now that you have a basic understanding of kernel drivers, let's dive into the structure of malicious kernel drivers, which are more commonly known as rootkits.

Rootkit Components

Rootkits commonly have two components: a process running in user space and a kernel driver that receives instructions from that process. Rootkits nearly always start with a user-space executable that must be deployed and executed on the victim host. Once this is accomplished, the malicious process loads a driver into kernel space. **Figure 14-2** shows a simple, high-level view of how rootkits are installed.

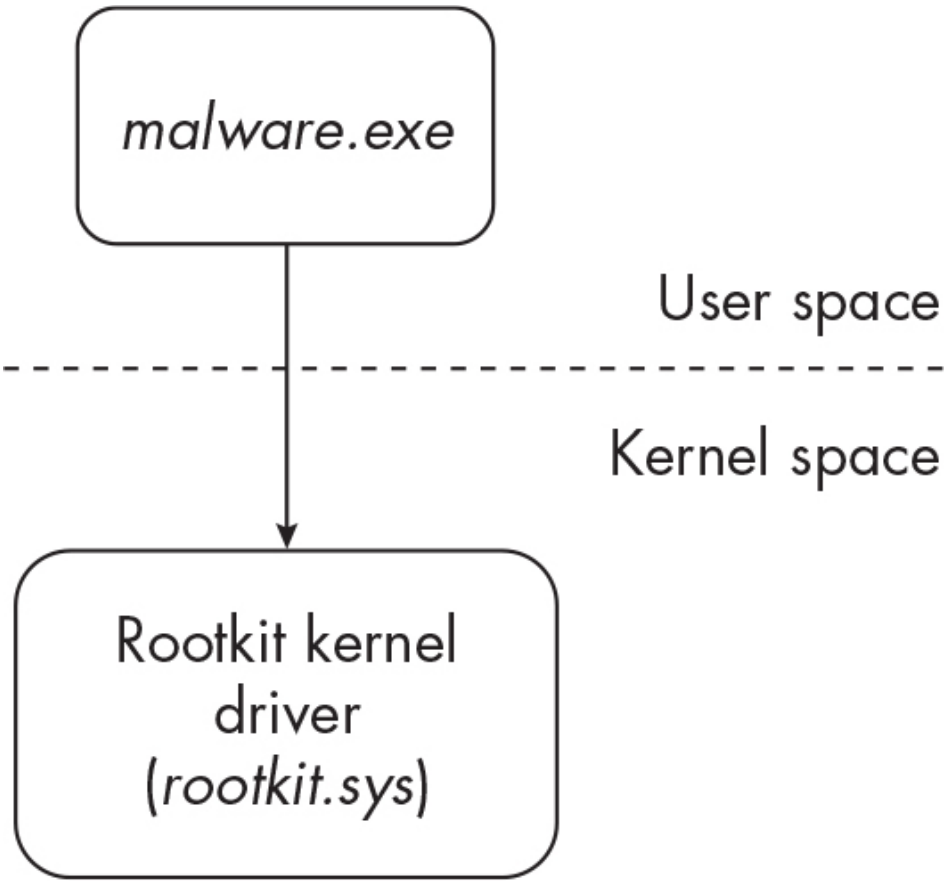

Figure 14-2: The rootkit installation process

First, the victim is delivered a dropper executable (*malware.exe*) that, once executed, decrypts an embedded malicious kernel driver (*rootkit.sys*). The dropper configures and executes this driver as a service, completing the rootkit's installation into kernel space. (We'll discuss this more in a moment.) The user-space process code contains the majority of the malware's primary functionalities, while the kernel component works

to mask and protect the user-space process on the system, establish low-level hooks to hide its artifacts in memory and on disk, and blind endpoint defenses and investigators to its presence.

Figure 14-3 illustrates some of the newly installed rootkit's functionalities.

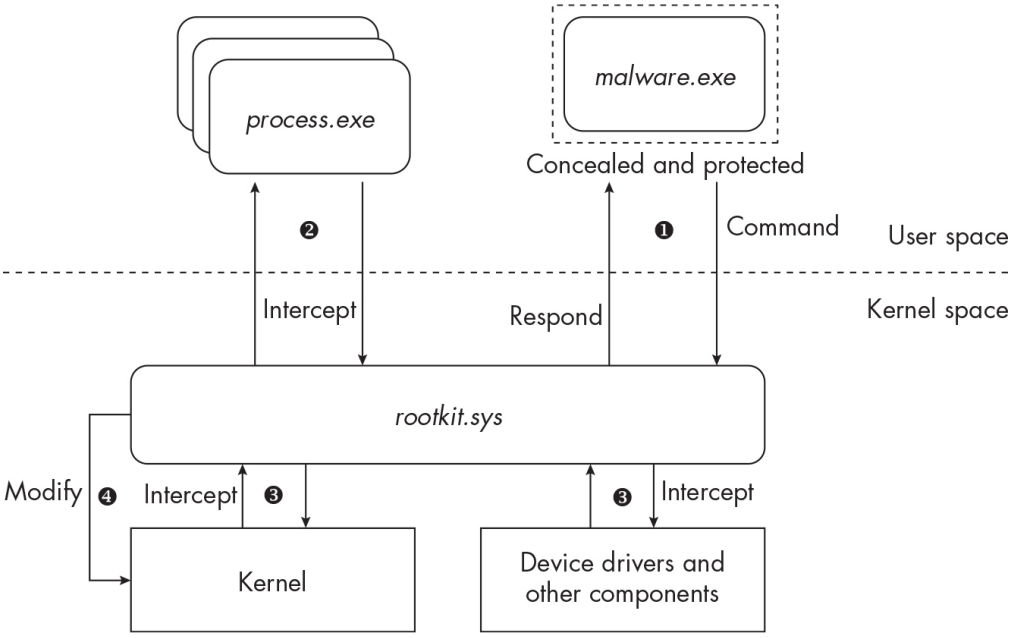

Figure 14-3: The basic functionalities of a rootkit

This rootkit can conceal its user-space executable (*malware.exe*) as well as issue commands to its kernel driver (*rootkit.sys*) ❶. Rootkit user-space components often communicate to their kernel-space counterparts by sending requests via a WinAPI function such as `DeviceIOControl`, which allows a user-space process to send a control code (instruction) to a kernel-space driver. In addition, this rootkit is able to hook and intercept API calls from user space ❷, intercept communication between other kernel components and device drivers ❸, and even tamper with kernel memory directly ❹. All of these techniques will become clearer as we progress through this chapter. But first, let's take a step back and talk about how rootkits are installed in the first place.

Rootkit Installation

In the past, rootkits were more prevalent, but Microsoft has implemented protective measures in later versions of Windows that make it more difficult to implement the changes necessary to install malicious kernel com-

ponents on a system. Even so, there are ways to bypass these protections, so these kinds of attacks do still happen sometimes. Let's take a look at a recent real-world example: HermeticWiper.

HermeticWiper targeted victims in Ukraine in 2022. It's not a rootkit per se; rather, it is destructive malware that requires low-level access to overwrite data on the disk, rendering a system unbootable. However, because HermeticWiper uses a common method of loading a kernel driver and is well documented, it's a good example of how rootkits can also be installed in a victim environment.

One particular HermeticWiper sample was signed by a certificate stolen from a valid company, Hermetica Digital Ltd., potentially allowing HermeticWiper to bypass certain endpoint defenses. (This technique was discussed in **Chapter 13**.) When HermeticWiper first executes, the sample writes a new *.sys* file with a file name consisting of four characters (such as *bpdr.sys*) to disk. This file is a legitimate driver from the company EaseUS that is normally used to resize and partition disks; however, it can be misused, as we'll see in a moment. Since at the time of the attack, this file is signed by a valid certificate, it's able to bypass Windows protections like driver-signing enforcement.

Next, to obtain the special privileges required for loading drivers, HermeticWiper attempts to obtain `SeLoadDriverPrivilege`. This privilege can be obtained only by a process already running at a high privilege level, so most malware will need to use a privilege elevation technique (such as a UAC bypass) to get administrator or system-level privileges and then call a function such as `AdjustTokenPrivileges` (as discussed in **Chapter 13**). Once it has the required privileges, HermeticWiper creates a new service by calling `CreateServiceW` and starts it by calling `StartServiceW`. Creating and executing a service is one of the most common methods of loading a new kernel driver, for both legitimate and malicious purposes. It can also be accomplished via the Windows command line, like so:

```
C:\> sc create "evil" binPath="C:\Users\Public\evil.sys" type=kernel
```

This command creates a new service (`evil`), specifying an input parameter of `C:\Users\Public\evil.sys` (the path of the malicious driver to be loaded)

and a type of `kernel`, denoting this service as a kernel driver installation. The following command can then be used to start the service:

```
C:\> sc start "evil"
```

Here's the output of executing these commands in Windows:

```
C:\Windows\system32> sc create "evil" binPath="C:\Users\Public\evil.sys" type=ker

[SC] CreateService SUCCESS

C:\Windows\system32> sc start "evil"

SERVICE_NAME: evil
TYPE               : 1 KERNEL_DRIVER
STATE              : 4 RUNNING
--snip--
WIN32_EXIT_CODE    : 0 (0x0)
--snip--
```

In the case of HermeticWiper, since the driver (at the time of attack, at least) is legitimate and signed by a valid authority, it likely won't have any issues installing in kernel space and can circumvent built-in Windows controls. If the driver wasn't signed by a valid signing authority, we'd receive the following error upon starting the service:

```
C:\Windows\system32> sc start "evil"

[SC] StartService FAILED 1275:
This driver has been blocked from loading
```

Once the malicious service is successfully installed, the Windows Service Controller takes over and loads the driver into kernel address space. The malware's user-space component can now interact with the malicious driver in kernel space by invoking `DeviceIOControl` and issuing commands to it. In doing so, HermeticWiper is using an otherwise legitimate driver (the EaseUS driver) to write data to the disk, destroying this data and making infected systems inoperable.

NOTE

Kernel drivers aren't always loaded using services. There are other techniques for loading them, including invoking the NT API function `NtLoadDriver`.

This abuse of legitimate drivers is a form of the Bring Your Own Vulnerable Driver technique, which we'll discuss next.

EXPLORING HERMETICWIPER

I would encourage you to explore HermeticWiper in your analysis VM. You can find a sample on VirusTotal or Malshare (`SHA256: 1bc44eef75779e3ca1eefb8ff5a64807dbc942b1e4a2672d77b9f6928d292591`).

Detonate the sample with Administrator privileges, and make sure to capture its behaviors in Procmon. See if you can locate where HermeticWiper is writing the driver file to disk, how it loads the driver, and how it interacts with the driver (via `DeviceIOControl` codes).

If you'd like to read more on HermeticWiper, see the following articles:

- Desai, Deepen, and Brett Stone-Gloss. "HermeticWiper & Resurgence of Targeted Attacks on Ukraine," *Zscaler*, February 24, 2022. *https://www.zscaler.com/blogs/security-research/hermeticwiper-resurgence-targeted-attacks-ukraine*.
- Editor. "ESET Research: Ukraine Hit by Destructive Attacks Before and During the Russian Invasion with HermeticWiper and IsaacWiper," *ESET*, March 1, 2022. *https://www.eset.com/int/about/newsroom/press-releases/research/eset-research-ukraine-hit-by-destructive-attacks-before-and-during-the-russian-invasion-with-hermet/*.
- Hasherezade, Ankur Saini, and Roberto Santos. "HermeticWiper: A Detailed Analysis of the Destructive Malware That Targeted Ukraine." *Malwarebytes*, March 4, 2022. *https://www.malwarebytes.com/blog/threat-intelligence/2022/03/hermeticwiper-a-detailed-analysis-of-the-destructive-malware-that-targeted-ukraine*.
- Roccia, Thomas. "Security Infographics: Overview of HermeticWiper," *SecurityBreak*, August 29, 2020. *https://blog.securitybreak.io*.

BYOVD Attacks

Bring Your Own Vulnerable Driver (*BYOVD*, or simply *BYOD*) attacks take advantage of legitimate, signed drivers as a sort of proxy to interact with the kernel; disable security controls; or load a separate, unsigned, malicious kernel driver. A malware author searches for a legitimate driver that is already signed by a valid signing authority (and therefore vetted by the Windows operating system) that can be dropped to the victim system during the attack. This driver must also have some sort of vulnerability that allows the threat actor to perform low-level malicious actions on the victim system. To exploit these vulnerabilities, rootkits often send commands to the vulnerable driver (by invoking `DeviceIOControl`, for example) from their user-space process.

One notable example of this type of attack is the FudModule rootkit, purportedly used by North Korean cybercriminals known as the Lazarus Group. As reported by researchers at ESET, FudModule takes advantage of a vulnerable, signed Dell driver containing a vulnerability (CVE-2021-21551) that allowed Lazarus to write data into kernel memory. More specifically, the vulnerability was triggered by a specially crafted instruction to the driver via `DeviceIOControl`. Ultimately, the threat actors successfully disabled multiple defensive mechanisms in Windows, effectively blinding endpoint defenses to later stages of the attack. For more information, see Peter Kálnai's article "Amazon-Themed Campaigns of Lazarus in the Netherlands and Belgium" at ***https://www.welivesecurity.com/2022/09/30/amazon-themed-campaigns-lazarus-netherlands-belgium/***.

Another example of malware that uses the BYOVD technique is the BlackByte ransomware family. As Sophos reported, BlackByte abuses a vulnerable driver in the legitimate product MSI AfterBurner, a tool for tuning graphics cards. The driver vulnerability (CVE-2019-16098) allowed the BlackByte operators to interact with the kernel and disable EDR products on the host by terminating EDR-related processes. To learn more about this malware, check out Andreas Klopsch's article "Remove All the Callbacks—BlackByte Ransomware Disables EDR Via RTCore64.sys Abuse" at ***https://news.sophos.com/en-us/2022/10/04/blackbyte-ransomware-returns/***.

A third example is the malware family ZeroCleare, found by researchers at IBM X-Force IRIS to be abusing a vulnerable VirtualBox driver (*vboxdrv.sys*), which allowed the attacker to execute shellcode in kernel memory and install a malicious kernel driver. You can read more about this attack in the IBM report "New Destructive Wiper ZeroCleare Targets Energy Sector in the Middle East" at ***https://securityintelligence.com/posts/new-destructive-wiper-zerocleare-targets-energy-sector-in-the-middle-east/***.

Unfortunately, there are other recent examples of malware abusing vulnerable drivers to disable and blind endpoint defenses, load additional malicious kernel drivers, or otherwise execute malicious code in privileged areas of the operating system. Furthermore, since these drivers are legitimate and signed, there's currently not much that can be done to completely prevent this type of attack. There's a dedicated project for tracking these vulnerable drivers; it's called Living Off The Land Drivers (LOLDrivers, for short). It's worth exploring if you're interested in learning more about BYOVD attacks, so visit the project website at ***https://www.loldrivers.io***.

NOTE

Not all BYOVD usage is rootkit related. For example, some malware simply leverages a vulnerable driver to execute kernel functions or perform low-level actions that would otherwise be prohibited.

Now that you've gotten an overview of how threat actors can bypass Windows protections to install rootkits, let's start looking into how rootkits behave on a victim host and manipulate the system to stay hidden. We'll start with an old technique: DKOM.

Direct Kernel Object Manipulation

Direct kernel object manipulation (*DKOM)* involves directly modifying data in kernel memory. This is a delicate task because, when done incorrectly, it can crash the operating system. Done correctly, however, it can give the malware immense power. One example of DKOM is hiding processes.

Using DKOM, a rootkit can hide its user-space processes and kernel modules from endpoint defenses and forensics analysts by modifying its processes' EPROCESS data structures in kernel memory. You might remember from **Chapter 1** that EPROCESS structures form a doubly linked list of processes running on the host. Some defense and analysis tools rely on these structures to monitor and inspect anomalous running processes.

To perform this type of DKOM technique, a malicious kernel-space module invokes a function such as `PsLookupProcessByProcessID` to get a pointer to its own user-space component's EPROCESS structure. Then, the malware can modify the *forward link (flink)* and *backward link (blink)* members of the EPROCESS structure, unlinking the structure from the EPROCESS chain. **Figure 14-4** illustrates normal, unmodified EPROCESS structures before unlinking.

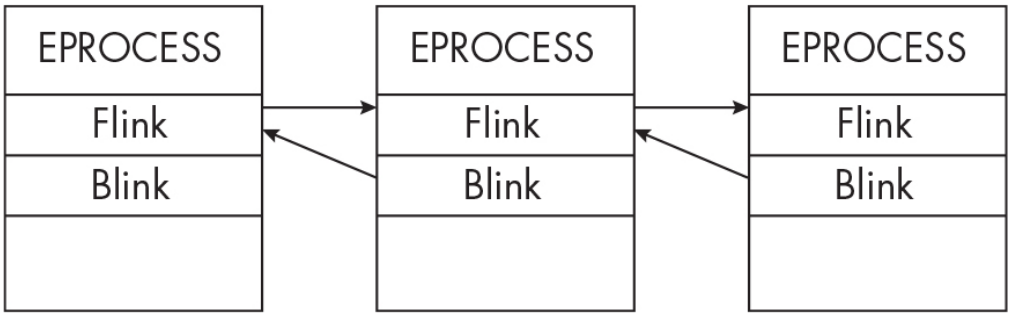

Figure 14-4: Doubly linked EPROCESS structures before unlinking

Notice how the EPROCESS structures are linked by their flink and blink members.

Figure 14-5 shows what happens when a rootkit tampers with the EPROCESS structures to unlink its malicious process (center).

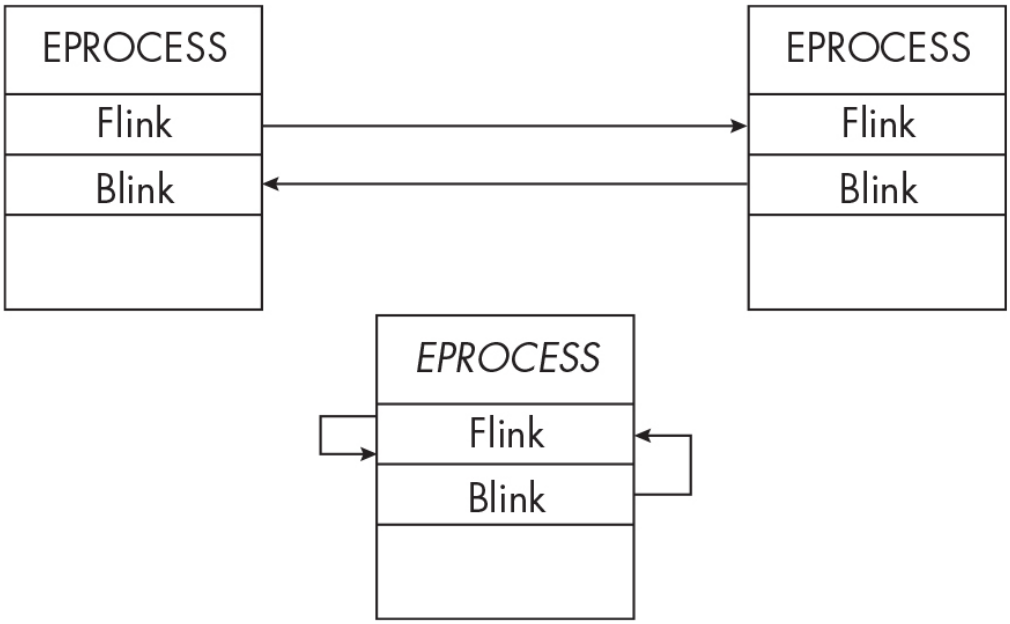

Figure 14-5: EPROCESS structures after unlinking

Notice how both the flink and blink for the malicious process's EPROCESS structure point back to it, effectively disconnecting this process from the normal EPROCESS chain.

The following code is an abridged version of the HideProcess project (see ***https://github.com/landhb/HideProcess***), and it demonstrates how the malware accomplishes the tasks outlined previously:

```
void remove_links(PLIST_ENTRY Current) {

    PLIST_ENTRY Previous, Next;
  ❶ Previous = (Current->Blink);
  ❷ Next = (Current->Flink);

    // Loop over self (connect previous with next).
    Previous->Flink = Next;
    Next->Blink = Previous;

    // Rewrite the current LIST_ENTRY to point to itself.
  ❸ Current->Blink = (PLIST_ENTRY)&Current->Flink;
  ❹ Current->Flink = (PLIST_ENTRY)&Current->Flink;
--snip--
```

First, the code defines the variables `Previous` (which stores the current process's blink pointer) ❶ and `Next` (which stores the current process's

flink pointer) ❷. Later, the code rewrites these `LIST_ENTRY` values to point to itself. The `Current->Blink = (PLIST_ENTRY)&Current->Flink` line sets the process's current blink pointer to its flink pointer value ❸. The `Current->Flink = (PLIST _ENTRY)&Current->Flink` line ensures that the process's flink points to itself ❹. In essence, this breaks the EPROCESS chain, hiding the process from process management tools (like Task Manager) and some forensics investigation toolsets, perhaps helping it better evade endpoint defenses.

DKOM isn't limited to hiding processes, however. Using DKOM techniques, malware can theoretically alter any object in kernel memory. Malware has been known to use DKOM techniques to hide malicious network traffic or alter files in order to interfere with forensics investigations. DKOM can also be used to inject kernel hooks, as you'll see in the next section.

While DKOM is one of the most basic and straightforward methods rootkits can use to hide or to alter the system, it's not a golden ticket. DKOM and other kernel manipulation techniques can easily crash the operating system, potentially alerting the victim to the malware's presence. Not only that, but security measures like PatchGuard also can create challenges for malware authors, as we'll discuss later in the chapter.

"Legacy" Kernel Hooking

Just like malware running in user space can use inline and IAT hooking to monitor, intercept, and manipulate function calls, kernel-space malware can use several types of hooks to launch its attacks. We'll discuss some of the most prevalent, starting with the decades-old technique of SSDT hooking.

NOTE

The techniques discussed in this section were at one time some of the most common ones used by both rootkits and endpoint defense products alike. Much like DKOM, however, they're no longer popular thanks to the protections that Microsoft has implemented in modern versions of Windows. Still, it's important to gain a basic understanding of them since you may occasionally witness malware using these or similar tactics, and it'll also give you a better grasp of more modern rootkit techniques.

SSDT Hooks

The *System Service Descriptor Table (SSDT)* or *Dispatch Table* contains an array of syscall IDs and their corresponding pointers to kernel functions. (These differ between 32- and 64-bit operating systems, but we won't go into those specifics here.) As discussed in **Chapters 1** and **13**, when a user-space process invokes Windows API and NT API functions, the function eventually makes a syscall into the kernel to fulfill the request. Let's look at an example using `NtReadFile` to read a file on disk. Here's the basic sequence of steps that must occur:

1. A program in user space invokes `NtReadFile`. The program initiates a syscall, referencing the syscall ID that corresponds to the `NtReadFile` function.

2. The syscall triggers the processor to switch from user mode to kernel mode and passes the request and syscall ID to the syscall handler.

3. The syscall handler consults the SSDT to obtain the address of the kernel `NtReadFile` function (which is exported from *ntoskrnl.exe*) and then proceeds to execute the function.

4. Since the `NtReadFile` function is being invoked to read a file on disk, it must communicate with kernel drivers such as the disk driver stack. This is where the IO manager is engaged.

5. The IO manager sends instructions in the form of *IO request packets (IRPs)* to the appropriate drivers, which will carry out `NtReadFile`'s requested actions (such as reading the specific file on disk). I'll discuss the IO manager and IRPs later in this chapter.

6. Once the drivers process the request, the result is sent back to the original calling program in user space.

Now that you have a basic understanding of how the SSDT is used, you can see how malware could insert a hook into it to redirect requests to malicious code. **Figure 14-6** shows an example of this approach with the `NtReadFile` function.

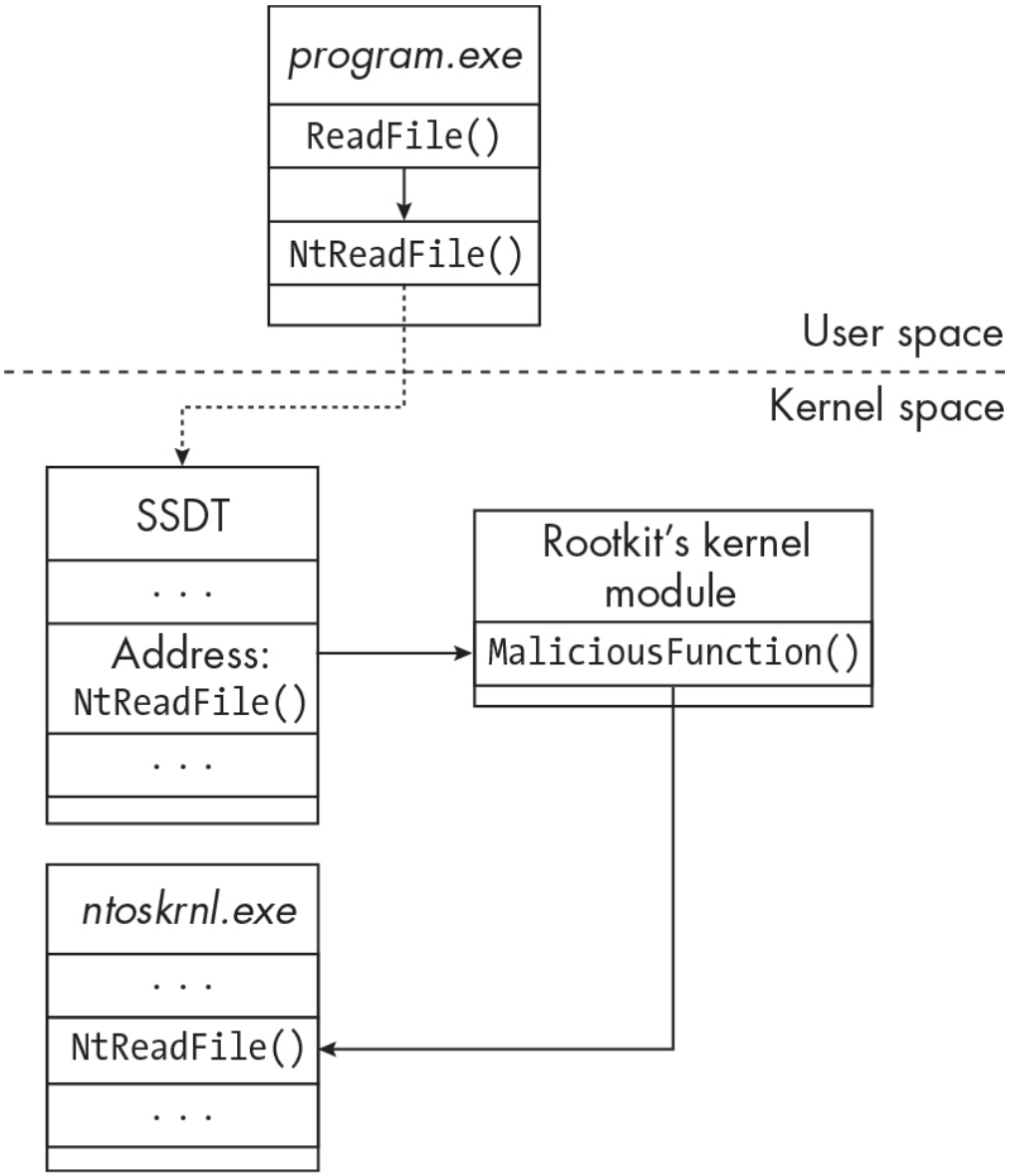

Figure 14-6: An SSDT hook for NtReadFile

The rootkit modifies the function pointer to `NtReadFile` inside the SSDT, which redirects the request for `NtReadFile` to the rootkit's malicious kernel module. The rootkit then intercepts and modifies the call to `NtReadFile`. Later, it can redirect the call to the original `NtReadFile` function code inside *ntoskrnl.exe*.

There are many reasons why a malware author would use SSDT hooking. For example, they might implement an SSDT hook for `NtReadFile` to prevent the malware's own malicious files and code from being read by endpoint defenses and investigators. Another kernel-hooking technique, inline kernel hooks, is used for a similar reason.

Inline Kernel Hooks

Inline hooking is a malware technique employed not just in user space (as discussed in **Chapter 12**) but in kernel functions as well. To install the hooks, a rootkit attempts to modify function code inside *ntoskrnl.exe*. Similar to the example just discussed with SSDT hooking, a rootkit could hook `NtReadFile` by tampering with the function's code to insert a jump instruction and redirect control flow to the malicious kernel module's code, as illustrated in **Figure 14-7**.

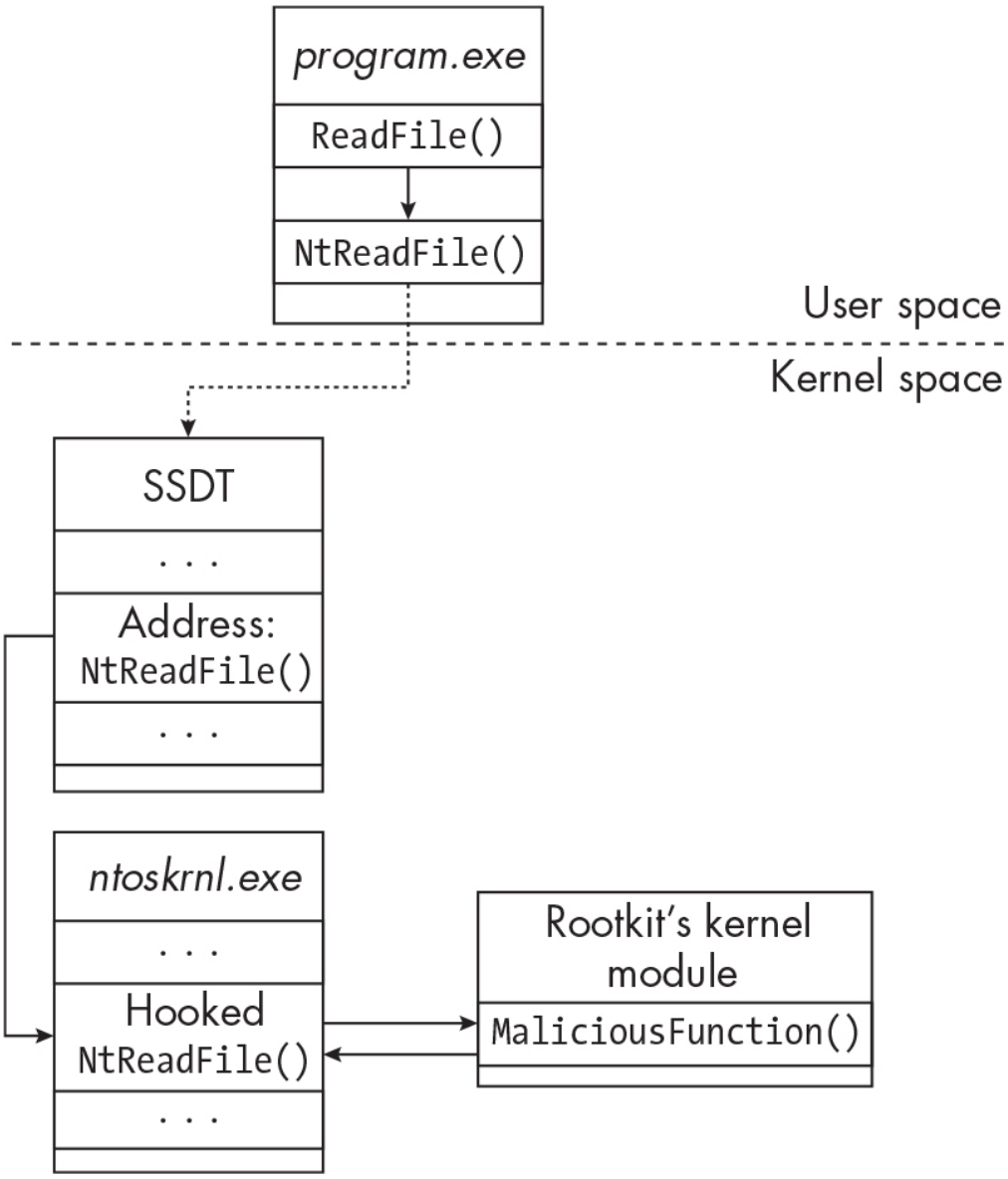

Figure 14-7: An inline kernel hook for NtReadFile

There are a number of ways a rootkit can write a hook into a target function. Similar to inline hooking in user space, the malware first must alter

the memory protections of the target function's code and then write a hook into it. The following pseudocode demonstrates how this might look:

```
// Set target memory to PAGE_READWRITE protection.
MmProtectMdlSystemAddress(mdl, "PAGE_READWRITE");

// Write a hook (a jump instruction) into the target function.
RtlCopyMemory(targetAddress, sourceAddress, size);

// Set the target memory back to original protections.
MmProtectMdlSystemAddress(mdl, "PAGE_READONLY");
```

`MmProtectMdlSystemAddress` is a kernel function that's used to set the memory protection type for a *memory descriptor list (MDL)*, which is a structure containing a memory address range. This function has two parameters: a `MemoryDescriptorList` (the memory address range that will be altered) and a protection constant (such as `PAGE_READWRITE`, which would change the MDL's protections to be writable).

Following this, the malware invokes a kernel function such as `RtlCopyMemory` to overwrite the target code with a jump instruction, for instance. The primary parameters of `RtlCopyMemory` are the destination address of the target memory region, the source address (which contains the jump instruction to be copied), and the size of the data being copied. Malware must be careful to set the target memory region back to its original protection setting because incorrect and anomalous protections (such as "writable") may raise the suspicions of endpoint defenses or cause system instability.

Next, we'll turn to IRP hooking, another type of kernel hook rootkits have been known to use.

IRP Hooks

When user-space programs need to communicate with kernel drivers, they do so via the IO manager. This communication is primarily accomplished with IO request packets (IRPs), which are objects comprising data structures that contain information about the request and actions to be performed. IRPs are passed between the calling program and kernel drivers, but they can also be used for communication between drivers. For ex-

ample, a USB keyboard driver will need to communicate with the USB host controller driver, and the IO manager helps facilitate this. This relationship is illustrated in **Figure 14-8**.

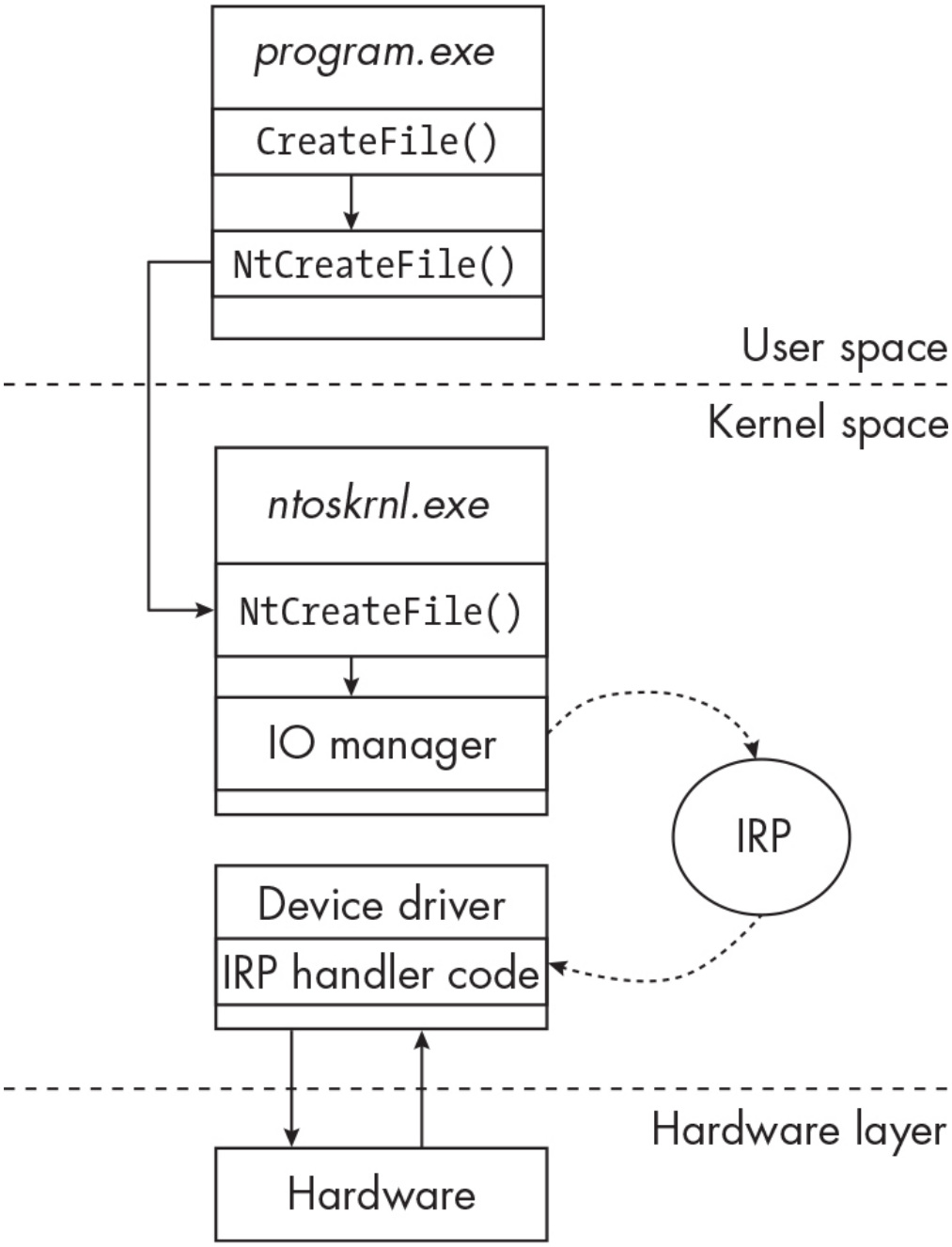

Figure 14-8: How an IRP works

This program invokes the `CreateFile` function to open a handle to a file on disk. Eventually, the program makes a syscall into the kernel (*ntoskrnl.exe*). This is where the IO manager gets involved, sending an IRP to the filesystem driver that will handle this operation. In the final step (not pictured here), the driver sends the status of the operation back to the IO manager, which will return the status to the calling program.

Every IRP includes an *IRP code*, which tells the recipient driver what IRP handler should be used to handle the respective request. **Table 14-1** lists some of the more interesting IRP function codes for our purposes.

Table 14-1: IRP Codes

IRP code	Request description
`IRP_MJ_CREATE`	Sent to a driver when the requesting thread opens a handle to a device or file object, such as when making a call to `NtCreateFile`
`IRP_MJ_WRITE`	Sent to a driver when the requester wishes to transfer data, such as when writing data to a file
`IRP_MJ_READ`	Sent to a driver when the requester wishes to read data, such as from a file
`IRP_MJ_DEVICE_CONTROL`	Sent when a `DeviceIoControl` function is called (meaning a user-space process is sending a direct control code, or instruction, to a driver)
`IRP_MJ_SHUTDOWN`	Sent when a system shutdown has been initiated
`IRP_MJ_SYSTEM_CONTROL`	Sent when a user-space process requests system information via Windows Management Instrumentation (WMI)

Each driver installed in the kernel includes a table of IRP handlers called the *major function table* (or the *IRP function table*). Major function tables contain pointers to the handler code that will handle a particular IRP; this code might be located in the driver itself or inside another driver or module. The following output shows an IRP function table for the FLTMGR driver:

```
IRP_MJ_CREATE                   0xfffff8023674ca20   FLTMGR.SYS
IRP_MJ_CREATE_NAMED_PIPE        0xfffff8023674ca20   FLTMGR.SYS
IRP_MJ_CLOSE                    0xfffff80236713e60   FLTMGR.SYS
```

```
IRP_MJ_READ                0xfffff80236713e60   FLTMGR.SYS
IRP_MJ_WRITE               0xfffff80236713e60   FLTMGR.SYS
IRP_MJ_QUERY_INFORMATION   0xfffff80236713e60   FLTMGR.SYS
IRP_MJ_SET_INFORMATION     0xfffff80236713e60   FLTMGR.SYS
IRP_MJ_QUERY_EA            0xfffff80236713e60   FLTMGR.SYS
--snip--
```

This output was created with the help of Volatility, a memory forensics and analysis tool. Although not covered in this book, memory forensics techniques can be great additions to the malware analysis process, especially in the case of rootkits. For this specific example, I used the `driverirp` module in Volatility.

The first column in this output contains the IRP code. The second and third columns contain the pointer to the associated IRP handler function and the module containing the handler code, respectively. In this case, the driver points to handlers it contains.

To intercept, modify, and gain control of IO communication, malicious kernel drivers might attempt to hook IRPs. One reason for doing so is to hide and protect the malware's artifacts on the endpoint by intercepting IRP function calls that reference those artifacts on disk. The Autochk rootkit (`SHA256: 28924b6329f5410a5cca30f3530a3fb8a97c23c9509a192f2092cbdf139a91d8`) does exactly this: it hooks `IRP_MJ_CREATE` inside the FLTMGR driver to intercept IRPs referencing its malicious files on disk (as illustrated in **Figure 14-9**).

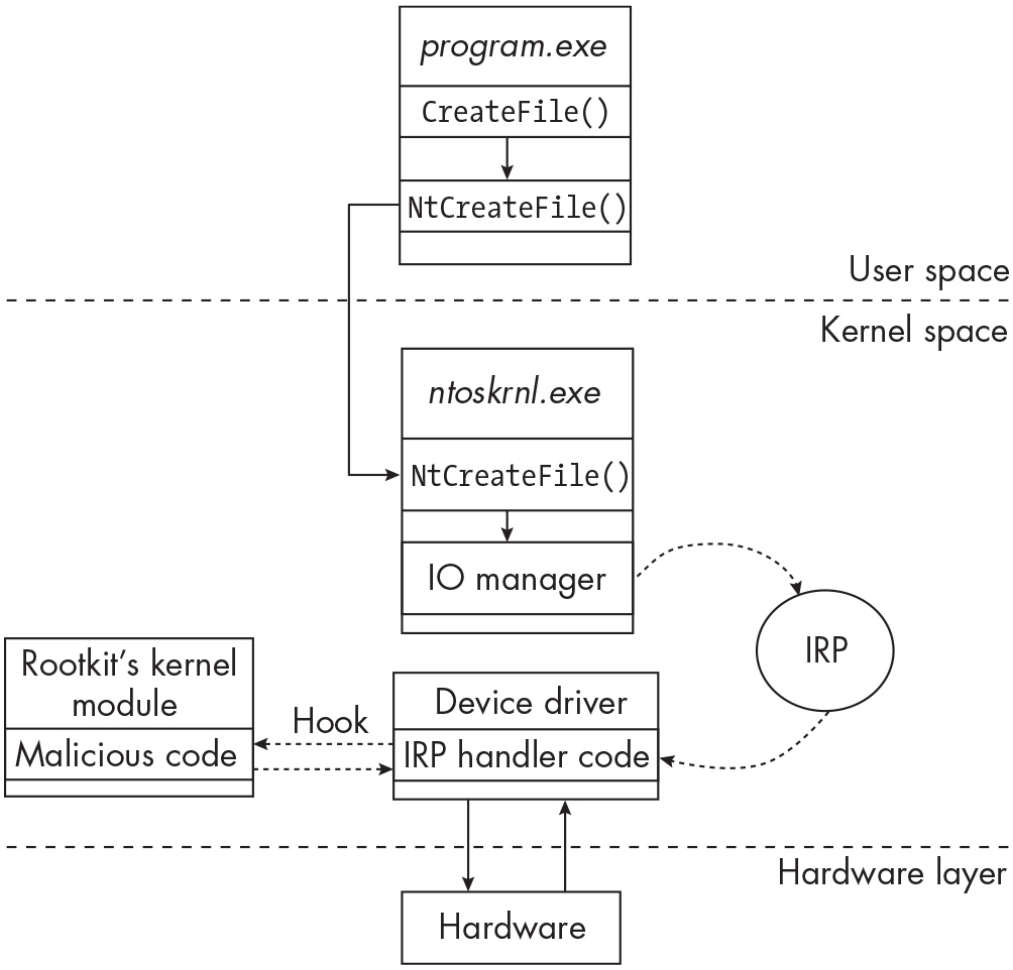

Figure 14-9: An IRP hook for NtReadFile

If an application in user space, such as a forensics tool, attempts to access this rootkit's files, the IRP will be handled by the rootkit's handler code rather than by the legitimate handler that would otherwise deal with this request. The following Volatility output shows the hooked FLTMGR driver's IRP function table:

```
0 IRP_MJ_CREATE                 0xfffff80230bf1bc4   autochk.sys
1 IRP_MJ_CREATE_NAMED_PIPE      0xfffff8023674ca20   FLTMGR.SYS
2 IRP_MJ_CLOSE                  0xfffff80236713e60   FLTMGR.SYS
3 IRP_MJ_READ                   0xfffff80236713e60   FLTMGR.SYS
4 IRP_MJ_WRITE                  0xfffff80236713e60   FLTMGR.SYS
5 IRP_MJ_QUERY_INFORMATION      0xfffff80236713e60   FLTMGR.SYS
6 IRP_MJ_SET_INFORMATION        0xfffff80236713e60   FLTMGR.SYS
7 IRP_MJ_QUERY_EA               0xfffff80236713e60   FLTMGR.SYS
--snip--
```

Notice anything shady? In the first row of the code, `FLTMGR.SYS` has been replaced with `autochk.sys`. All `MJ_CREATE` IRPs destined for the FLTMGR driver

will instead be forwarded to the malicious handler code inside the rootkit's driver, `autochk.sys`! You can read more about some of the techniques of this rootkit at ***https://repnz.github.io/posts/autochk-rootkit-analysis/***.

To install an IRP hook, malware authors have a few options. One approach is to replace the original handler code pointer value in a victim driver with a pointer to malicious handler code. Alternatively, malware could use the inline hooking method described previously to overwrite the first few bytes in the legitimate handler function with a jump instruction to malicious code.

Both of these techniques, as well as the other hooking techniques mentioned, rely on the delicate task of manipulating kernel objects in memory. As noted earlier, however, the techniques discussed in this section aren't often used in malware anymore due to the protections now built into Windows. With this in mind, let's shift to some relatively modern techniques that rootkits might use to circumvent these Windows protections, starting with IRP filtering and interception.

IRP Interception by Filtering

Rather than crudely hooking kernel drivers to intercept and manipulate IRPs, rootkits can register a filter or minifilter driver to do so. Introduced at the beginning of this chapter, filter drivers and minifilter drivers can be "attached" to a device and added to its driver stack, intercepting IRPs as they are filtered down the stack. Let's go over this process in more detail.

NOTE

Filter drivers (sometimes called legacy filter drivers*) and minifilter drivers are both types of filters, but they operate quite differently. I won't go into the specifics of these two drivers in this book.*

Each hardware device attached to the system has an associated hierarchical stack of drivers that enables communication between the device and the operating system (see **Figure 14-10**).

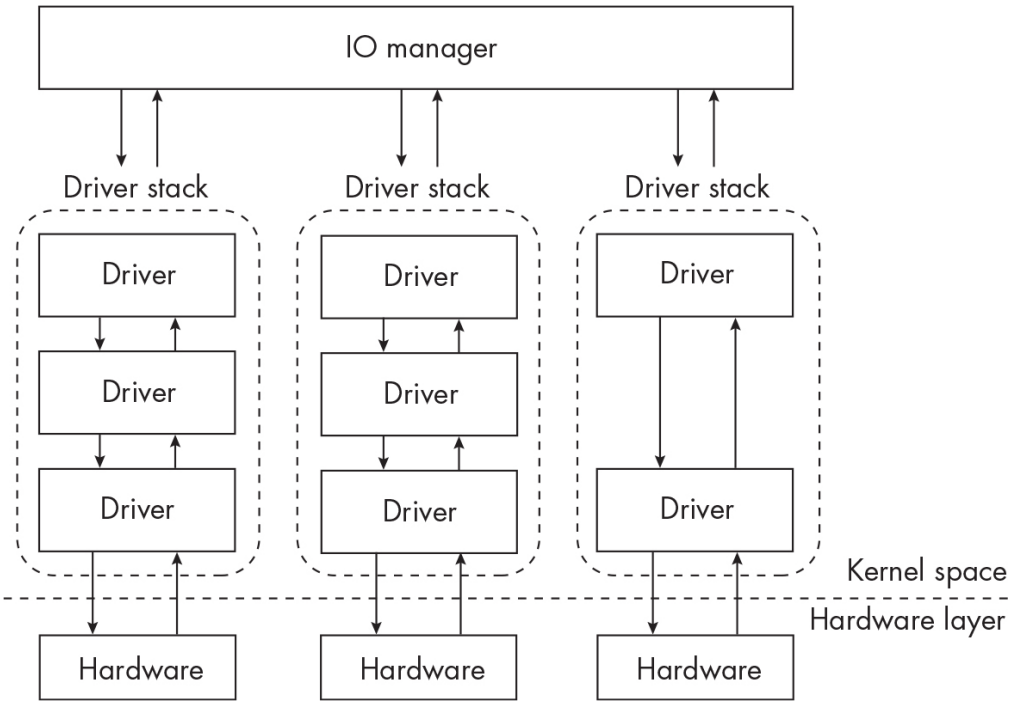

Figure 14-10: How the IO manager communicates with driver stacks

Each driver in the stack performs a specific role and serves as an interface between the drivers before and after it in the stack. Additionally, when the IO manager sends an IRP to a specific device, the IRP is routed through the device's hierarchical driver stack, passing through each driver in the stack one by one. If one of the drivers has a handler for the specific IRP, it takes some sort of action on that IRP. The arrows shown in **Figure 14-10** represent communication between drivers in the form of IRPs.

Filter drivers are designed to be inserted into a driver stack to add functionality. They can be inserted in various locations (called *altitudes*) in the stack or even added all the way to the top of the stack, where they can intercept any and all IRPs destined for the driver stack (see **Figure 14-11**).

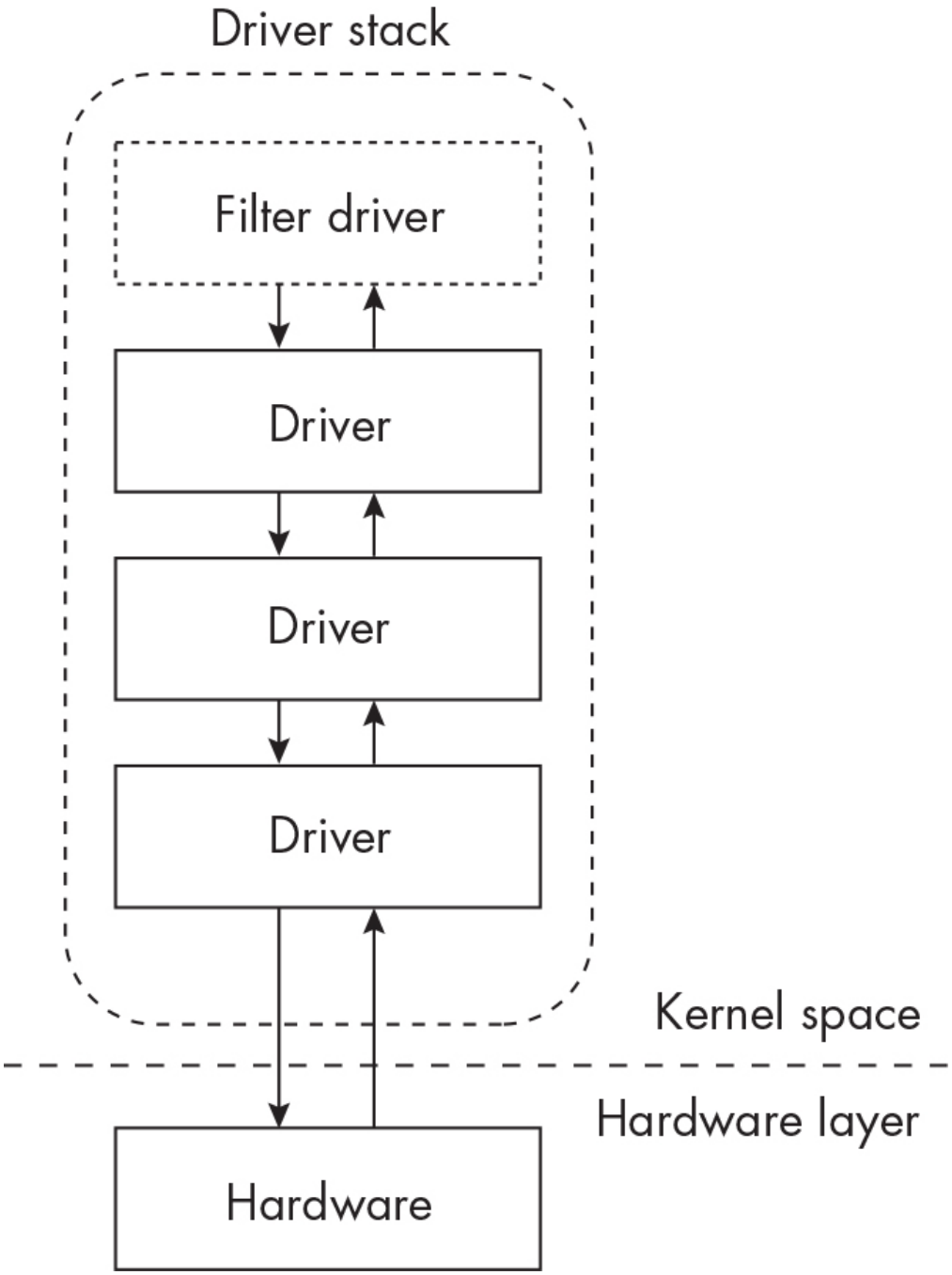

Figure 14-11: A filter driver added to the top of the driver stack

Rootkits can register a new filter driver and "insert" it at the top of a driver stack, allowing them to see and intercept all inbound IRPs. After intercepting, the malware can choose to drop the IRP or modify it. A rootkit might use a filter driver this way to protect its own files. For example, it could register a filter driver to watch for and intercept IO requests to its own files, either modifying the requests or dropping them completely to effectively hide them from investigators and analysis tools. EDR and other endpoint defenses sometimes use filter drivers for the same reason: to protect their files from malware.

To implement a filter driver, a rootkit must first load the driver into kernel memory (potentially using the techniques from "Rootkit Installation" on **page 269**). The filter driver must specify which IRP communication it cares about by setting up a handler for those IRPs. For instance, if the malware is trying to intercept `MJ_CREATE` IRPs, it must implement this in the filter driver's IRP function table.

Malware can abuse minifilter drivers by registering one of its own (by calling `FltRegisterFilter`) or by hooking an existing minifilter. Some modern malware has been known to do this. When analyzing rootkit driver code, take note of whether the malware is calling `FltRegisterFilter` to register its own minifilter driver. Also notice whether the malware is calling functions such as `FltGetFilterFromName`, `FltEnumerateFilters`, or `FltEnumerateInstances`; it may be attempting to enumerate other minifilters on the host, preparing for hooking. For more information on how minifilter drivers are implemented in practice, see Rahul Dev Tripathi's article "Storage Device Restriction Using a Minifilter Driver Approach" at ***https://www.codeproject.com/Articles/5341729/Storage-Device-Restriction-Using-a-Minifilter-Driv***.

Legacy filter drivers are installed differently. The specifics are outside the scope of this book, but you can learn more from the filesystem driver tutorial at ***https://www.codeproject.com/Articles/43586/File-System-Filter-Driver-Tutorial*** and from Rotem Salinas's great write-up "Fantastic Rootkits and Where to Find Them" at ***https://www.cyberark.com/resources/threat-research-blog/fantastic-rootkits-and-where-to-find-them-part-1***.

Abusing Kernel Callbacks

Abusing kernel callbacks is another more modern approach used by some rootkits. To recap the discussion in **Chapter 13**, a callback allows a kernel module to be notified of system events so that it can take some sort of action when they happen. For example, a driver may need to know when a process executed in user space, so it would implement the `PsSetCreateProcessNotifyRoutine` callback (as in the case of some EDR products). Once this callback is registered with the driver, the driver will receive a notification in the form of an IRP when a process is created on the

system, giving the driver the chance to execute its callback code for that event.

The creator of the process is responsible for sending out a notification to all registered drivers. So, when a process spawns a child process, for example, the calling process sends out the `CreateProcessNotifyRoutine` notification to all registered drivers. When a driver receives the notification, the driver's callback code will be executed.

A rootkit can also use callbacks if it wants to be notified of specific system events. Once an event occurs, such as a registry modification or filesystem operation, the rootkit's malicious driver(s) will be notified, and the callback code will be executed. The following output shows a listing of registered callback routines on an infected system:

```
Type                                Callback           Module
-----------------------------------------------------------------------
IoRegisterShutdownNotification     0xfffff8033891e830  ntoskrnl.exe
IoRegisterShutdownNotification     0xfffff8033b6b10c0  SgrmAgent.sys
IoRegisterShutdownNotification     0xfffff803390cf320  ntoskrnl.exe
--snip--
PsRemoveLoadImageNotifyRoutine     0xfffff80af3afb210  ahcache.sys
PsRemoveCreateThreadNotifyRoutine  0xfffff80336bd1060  mmcss.sys
PsSetCreateThreadNotifyRoutine     0xfffff6050d26ccc0  comp.sys
KeBugCheckCallbackListHead         0xfffff8033c13cb90  ndis.sys
KeBugCheckCallbackListHead         0xfffff8033c59b4e0  fvevol.sys
--snip--
```

In this output, the `Type` column shows the callback type, the `Callback` column shows the address of the callback handler, and the `Module` column shows the kernel module that registered the callback. Most of these are normal, legitimate callbacks. However, there's a suspicious module name (`comp.sys`) that appears to have registered an interesting callback (`PsSetCreateThreadNotifyRoutine`). As mentioned previously, this callback will trigger when a new thread is created by a process in user space. Also note that the address of the callback code is much different from those of the legitimate callbacks (`0xfffff6050d26ccc0` versus `0xfffff80af3afb210`, for example).

A similar approach was taken by the DirtyMoe rootkit. DirtyMoe used kernel callbacks to silently inject malicious code into newly created threads in user space. You can read more about it in Martin Chlumecký's article "DirtyMoe: Rootkit Driver" at ***https://decoded.avast.io/martinchlumecky/dirtymoe-rootkit-driver/***.

The infamous Necurs rootkit, which originated in 2014, sets up a registry callback (`CmRegisterCallback`), a type of filter driver callback that will notify it of any access to its registry service key. If an investigator or program attempts to access this registry key, the attempt fails. This simplified pseudocode example shows how a malicious driver could register and abuse a registry callback:

```
❶ void RegistryCallback(..., ..., context)
  {
    if (context)
    {
     ❷ if (event == CM_EVENT_REGISTRY_KEY_OPEN)
       {
        ❸ if (context->registryKey == "HKEY_CURRENT_USER\Software\Microsoft\
  Windows\CurrentVersion\evil")
         {
        ❹ // Block the action.
         }
       }
    }
  }

❺ CmRegisterCallback(RegistryCallback, &context);
```

This malware code first defines the callback function code (`RegistryCallback`) that will be executed once the callback occurs ❶. Later in the code, the rootkit defines the registry callback, passing the callback name (`RegistryCallback`) and also the context, which is a pointer to a structure containing information about the function call ❺. Since this callback will be triggered by programs interacting with the Windows registry, this context structure contains important information like the particular registry action (open key, write data, and so on) and the target of the action (or the specific registry key or value affected).

When a program performs a registry action, such as invoking `RegOpenKeyExA`, the rootkit's malicious callback code will be executed. The rootkit checks to see if the registry event is equal to `CM_EVENT_REGISTRY_KEY_OPEN` (indicating that a registry key is being opened) ❷ and then checks to see whether the registry key being acted upon is `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\evil` (the key used by the malware to establish persistence on the host) ❸. If the key name matches, the rootkit attempts to prevent the program or investigator from inspecting that registry key ❹. It can do so by temporarily deleting its own registry key and re-creating it later, or by injecting malicious code into the calling process and hooking into the function call to prevent the call from succeeding, among other methods.

NOTE

`CmRegisterCallback` is now obsolete; the modern version of this function is `CmRegisterCallbackEx`. The principles of the function remain the same, however.

You've seen quite a bit about how rootkits operate at a very low level in the operating system to manipulate kernel memory, install hooks, and configure callbacks, allowing them to remain hidden and evade defenses. Now we'll look briefly at a variant of malware that delves even deeper: bootkits.

Bootkits

A *bootkit* is a piece of malware designed to hide inside the system firmware, compromising the entire boot process. If a bootkit is able to tamper with the operating system boot-up, injecting itself into this process chain, it can effectively achieve all the benefits of a rootkit while also surviving system rebuilds.

One specific type of bootkit is a UEFI bootkit (sometimes called a UEFI rootkit), which operates within the *Unified Extensible Firmware Interface (UEFI)*, a specialized storage chip attached to a system's motherboard. The UEFI contains low-level software that executes before the operating system boots up, providing an interface between the operating system kernel and the various firmware devices installed in the system. Given that

the UEFI boots before the operating system, malware that can embed itself within the UEFI chip will remain undetected for longer periods of time and can even survive operating system reinstallations and rebuilds.

One notable example of a UEFI bootkit is CosmicStrand. In July 2022, researchers from Kaspersky reported that this UEFI bootkit dug itself into systems, with the entry vector possibly being a hardware vulnerability. The bootkit affected various systems with certain models of Asus and Gigabyte motherboards and took control of the Windows operating system kernel loader, injecting malicious code into kernel memory. For more about this threat, see the article "CosmicStrand: The Discovery of a Sophisticated UEFI Firmware Rootkit," from Kaspersky's Global Research & Analysis Team (GreAT) at ***https://securelist.com/cosmicstrand-uefi-firmware-rootkit/106973/***.

Another example is the MosaicRegressor framework, which was also discovered by Kaspersky. It included a UEFI rootkit component that hijacked the Windows boot process to drop an executable to disk that silently executes when Windows boots up. If this executable is removed from the disk, it will be rewritten to disk upon reboot of the system, providing a high degree of persistence. You can read the article from Kaspersky about MosaicRegressor, "Lurking in the Shadows of UEFI," by Mark Lechtik, Igor Kuznetsov, and Yury Parshin, at ***https://securelist.com/mosaicregressor/98849/***.

Compared with traditional user-space malware, bootkits are relatively rare. However, they might not be as rare as they're perceived to be. Because of their low-level access to the host, they can survive and persist undetected even in well-defended environments. If we can't detect this type of malware, we don't know it exists, which leads us to the unsettling conclusion that this type of malware could be embedded in more systems than we know. However, all is not lost. Let's wrap up this chapter by discussing some of the built-in Windows defenses against rootkits and bootkits.

Defenses Against Rootkits

Microsoft has implemented several defenses against rootkits, two of the most important being *PatchGuard* and *Driver Signature Enforcement*

(DSE). Introduced in 2005 for x64 versions of Windows XP, PatchGuard, which is also known as *Kernel Patch Protection (KPP)*, mitigates many of the rootkit techniques described earlier, such as SSDT and IDT hooking and many forms of DKOM. PatchGuard works by periodically verifying the integrity of kernel memory structures to test whether they've been modified. If PatchGuard detects that one of these structures has been modified, it forces a crash of the kernel, which has the result shown in **Figure 14-12**.

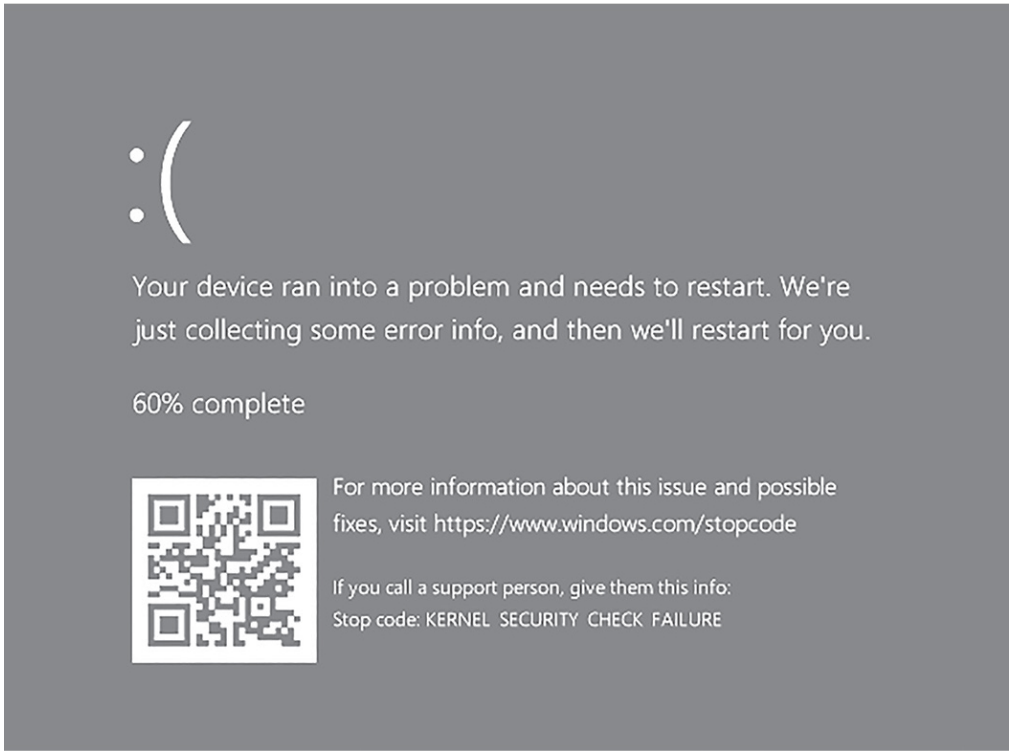

Figure 14-12: A kernel security check crash caused by PatchGuard

PatchGuard isn't impervious to circumvention, however. Since it scans kernel memory periodically, if these checks are timed properly, malware could very quickly tamper with kernel memory and then revert to a "clean" state before PatchGuard executes its integrity check. To initiate this check, the operating system calls the `KeBugCheckEx` kernel API function, which certain malware has been known to hook to prevent the kernel integrity check from successfully executing. There have also been several issues with malware exploiting PatchGuard and other related components. One example of such malware is GhostHook, which exploited a vulnerability in the way Windows implements a certain low-level Intel API called Intel Processor Trace, potentially allowing malware to fly under PatchGuard's radar. This attack technique is quite complex, so we won't

go into the details here, but you can read more about it in Kasif Dekel's post, "GhostHook—Bypassing PatchGuard with Processor Trace Based Hooking," at ***https://www.cyberark.com/resources/threat-research-blog/ghosthook-bypassing-patchguard-with-processor-trace-based-hooking***.

Two other relatively recent examples of malware that evade PatchGuard are InfinityHook (***https://github.com/everdox/InfinityHook***), which abuses a kernel API called `NtTraceEvent`, and ByePg (***https://github.com/can1357/ByePg***), which hijacks a kernel structure called the `HalPrivateDispatchTable`. Both of these circumvent PatchGuard in different ways. Note, however, that Microsoft has been quick to patch some of these known vulnerabilities in PatchGuard, forcing malware authors to adapt.

As mentioned at the beginning of this section, another security control Microsoft has implemented is Driver Signature Enforcement (DSE), sometimes called *digital signature enforcement*, which has been released for Windows Vista (x64) and more recent versions. DSE ensures that only pre-verified (signed) drivers are allowed to be loaded into kernel memory. In theory, legitimate drivers will be permitted, while suspicious, unsigned drivers will be prevented from loading. You read earlier in the chapter how malware can circumvent this control by using a malicious kernel driver signed with a legitimate certificate or by using BYOVD techniques. Microsoft recommends dealing with this problem by using *blocklists* of known vulnerable drivers. If a driver is reported to be vulnerable or is actively being misused, Microsoft adds it to the blocklist, which prevents it from being installed later. You can enforce this feature by enabling the "Microsoft Vulnerable Driver Blocklist" security option in later versions of Windows. The primary concerns with this control are that some legitimate drivers may be prevented from loading and that it protects only against known malicious drivers.

Finally, *early launch anti-malware (ELAM)* is a feature of some endpoint defense software that protects the Windows boot process. ELAM is responsible for loading anti-malware kernel components prior to other third-party components. This ensures that the anti-malware is properly loaded and running before rootkits or any other persistent malware have the opportunity to load and execute. ELAM can be a good defense against

rootkits. However, as ELAM drivers aren't loaded until later stages in the boot process, ELAM alone might not prevent loading of bootkits.

For defense against bootkits and UEFI rootkits, you can enable *Secure Boot*. Available on most modern hardware, Secure Boot prevents malicious code from hijacking the Windows boot process. Upon boot-up, Secure Boot verifies the integrity of UEFI firmware drivers and the operating system itself before allowing the system to fully boot. This provides a layer of protection in the event malware has embedded itself in a UEFI chip. Secure Boot is optional in most versions of Windows, but it's required in Windows 11. As with all security controls, however, various implementations of Secure Boot have vulnerabilities that could be exploited by malware. Researchers from Eclypsium (***https://eclypsium.com***) reported on some of these vulnerabilities in 2020 and 2022, for example.

As a final note, many Windows rootkit protections, such as PatchGuard and DSE, are for x64 (64-bit) versions of Windows only. This leaves x86 (32-bit) versions of Windows potentially exposed to a host of dangerous low-level malware. Fortunately, precisely because these security features aren't enabled in x86 mode, EDR and anti-malware can use these same techniques for good, to monitor and protect the endpoint.

Summary

This chapter covered the fundamentals of rootkits: how kernel modules work, how malware installs malicious modules, and how threat actors bypass protections such as signed driver enforcement. We discussed some common rootkit techniques such as DKOM, kernel hooking, IRP interception, and kernel callback abuse. You were introduced to bootkits and also saw how kernel-space malware can bypass built-in Windows protection mechanisms like PatchGuard. This chapter has only scratched the surface of rootkits and kernel manipulation techniques, however, so if you're interested in learning more, I encourage you to review **Appendix C** for more resources. In the next chapter, we'll discuss how modern malware evades endpoint defenses and investigators by leveraging "fileless" and anti- forensics techniques.